

Python: exercises on classes

This notebook was generated in **pseudocode** mode.

Exercise 1: Simple Person Class

Exercise Description

Create a class called Person with attributes name and age. Add a method called introduce that prints "Hello, my name is [name] and I am [age] years old.".

Example:

```
p = Person("Alice", 30)
p.introduce()
# Output: Hello, my name is Alice and I am 30 years old.
```

After creating the class, create a Person object with name "Alice" and age 30, and store it in a variable called result.

Your solution

```
class Person:
    # step 1: define the constructor method that takes self, name, and age as parameters
    # step 2: assign the name parameter to self.name
    # step 3: assign the age parameter to self.age

    # step 4: define the introduce method that takes self as parameter
    # step 5: print a formatted string with the person's name and age

# step 6: create a Person object with name "Alice" and age 30
# step 7: store the Person object in the result variable
```

Test your solution

```
# Test the Person class
assert hasattr(result, 'name'), "Person should have a name attribute"
assert hasattr(result, 'age'), "Person should have an age attribute"
assert result.name == "Alice", "Person name should be Alice"
assert result.age == 30, "Person age should be 30"
assert hasattr(result, 'introduce'), "Person should have an introduce method"
```

Test your solution

```
# Test creating different Person objects
p2 = Person("Bob", 25)
assert p2.name == "Bob"
assert p2.age == 25

# Test that objects are independent
```

```
assert result.name != p2.name
assert result.age != p2.age
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 2: Dog Class with Default Values

Exercise Description

Create a class called Dog with attributes name and breed. If no breed is provided, default it to "Unknown". Add a method bark that prints "Woof! I am a [breed]!".

Example:

```
d1 = Dog("Max", "Golden Retriever")
d1.bark()
# Output: Woof! I am a Golden Retriever!

d2 = Dog("Buddy")
```

```
d2.bark()  
# Output: Woof! I am an Unknown!
```

After creating the class, create two Dog objects: one with name "Max" and breed "Golden Retriever", and another with just name "Buddy". Store them in variables called result1 and result2.

Your solution

```
class Dog:  
    # step 1: define the constructor method that takes self, name, and breed with default breed  
    # step 2: assign the name parameter to self.name  
    # step 3: assign the breed parameter to self.breed  
  
    # step 4: define the bark method that takes self as parameter  
    # step 5: print a formatted string with "Woof! I am a" and the breed  
  
    # step 6: create a Dog object with name "Max" and breed "Golden Retriever"  
    # step 7: create a Dog object with just name "Buddy" (using default breed)  
    # step 8: store the Dog objects in result1 and result2 variables
```

Test your solution

```
# Test the Dog class with specified breed  
assert hasattr(result1, 'name'), "Dog should have a name attribute"  
assert hasattr(result1, 'breed'), "Dog should have a breed attribute"  
assert result1.name == "Max", "First dog name should be Max"  
assert result1.breed == "Golden Retriever", "First dog breed should be Golden Retriever"  
assert hasattr(result1, 'bark'), "Dog should have a bark method"
```

Test your solution

```
# Test the Dog class with default breed
assert result2.name == "Buddy", "Second dog name should be Buddy"
assert result2.breed == "Unknown", "Second dog breed should default to Unknown"

# Test that objects are independent
assert result1.name != result2.name
assert result1.breed != result2.breed
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 3: Rectangle Class with Methods

Exercise Description

Create a class `Rectangle` with attributes `length` and `width`. Add a method `area` that returns the area of the rectangle and a method `perimeter` that returns the perimeter.

Example:

```
r = Rectangle(5, 3)
print(r.area())      # Output: 15
print(r.perimeter()) # Output: 16
```

After creating the class, create a Rectangle object with length 5 and width 3, and store it in a variable called result.

Your solution

```
class Rectangle:
    # step 1: define the constructor method that takes self, length, and width as parameters
    # step 2: assign the length parameter to self.length
    # step 3: assign the width parameter to self.width

    # step 4: define the area method that takes self as parameter
    # step 5: return the product of length and width

    # step 6: define the perimeter method that takes self as parameter
    # step 7: return 2 times the sum of length and width

# step 8: create a Rectangle object with length 5 and width 3
# step 9: store the Rectangle object in the result variable
```

Test your solution

```
# Test the Rectangle class
assert hasattr(result, 'length'), "Rectangle should have a length attribute"
assert hasattr(result, 'width'), "Rectangle should have a width attribute"
assert result.length == 5, "Rectangle length should be 5"
```

```
assert result.width == 3, "Rectangle width should be 3"
assert hasattr(result, 'area'), "Rectangle should have an area method"
assert hasattr(result, 'perimeter'), "Rectangle should have a perimeter method"
```

Test your solution

```
# Test the methods
assert result.area() == 15, "Area should be 15 (5 * 3)"
assert result.perimeter() == 16, "Perimeter should be 16 (2 * (5 + 3))"

# Test with different rectangle
r2 = Rectangle(4, 6)
assert r2.area() == 24, "Area should be 24 (4 * 6)"
assert r2.perimeter() == 20, "Perimeter should be 20 (2 * (4 + 6))"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 4: Bank Account Class

Exercise Description

Create a class `BankAccount` with attributes `balance` (initialized to 0). Add methods `deposit` to add to the balance and `withdraw` to subtract from the balance.

Example:

```
account = BankAccount()  
account.deposit(100)  
account.withdraw(40)  
print(account.balance)  # Output: 60
```

After creating the class, create a `BankAccount` object, deposit 100, withdraw 40, and store the account in a variable called `result`.

Your solution

```
class BankAccount:
    # step 1: define the constructor method that takes self as parameter
    # step 2: initialize self.balance to 0

    # step 3: define the deposit method that takes self and amount as parameters
    # step 4: add the amount to self.balance

    # step 5: define the withdraw method that takes self and amount as parameters
    # step 6: subtract the amount from self.balance

# step 7: create a BankAccount object
# step 8: call deposit method with 100
# step 9: call withdraw method with 40
# step 10: store the BankAccount object in the result variable
```

Test your solution

```
# Test the BankAccount class
assert hasattr(result, 'balance'), "BankAccount should have a balance attribute"
assert hasattr(result, 'deposit'), "BankAccount should have a deposit method"
assert hasattr(result, 'withdraw'), "BankAccount should have a withdraw method"
assert result.balance == 60, "Balance should be 60 after deposit 100 and withdraw 40"
```

Test your solution

```
# Test with new account
account2 = BankAccount()
assert account2.balance == 0, "New account should start with balance 0"

account2.deposit(50)
```

```
assert account2.balance == 50, "Balance should be 50 after deposit"

account2.withdraw(20)
assert account2.balance == 30, "Balance should be 30 after withdraw"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 5: Private Bank Account Class

Exercise Description

Modify BankAccount to have a private balance attribute (e.g., `_balance`). Ensure users can only view or change the balance through `deposit`, `withdraw`, and `get_balance` methods.

Example:

```
account = BankAccount()
account.deposit(100)
account.withdraw(40)
print(account.get_balance()) # Output: 60
```

After creating the class, create a BankAccount object, deposit 100, withdraw 40, and store the account in a variable called result.

Your solution

```
class BankAccount:
    # step 1: define the constructor method that takes self as parameter
    # step 2: initialize self._balance to 0 (private attribute with underscore)

    # step 3: define the deposit method that takes self and amount as parameters
    # step 4: add the amount to self._balance

    # step 5: define the withdraw method that takes self and amount as parameters
    # step 6: subtract the amount from self._balance

    # step 7: define the get_balance method that takes self as parameter
    # step 8: return self._balance

# step 9: create a BankAccount object
# step 10: call deposit method with 100
# step 11: call withdraw method with 40
# step 12: store the BankAccount object in the result variable
```

Test your solution

```
# Test the BankAccount class with private balance
assert hasattr(result, '_balance'), "BankAccount should have a _balance private attri
assert hasattr(result, 'deposit'), "BankAccount should have a deposit method"
assert hasattr(result, 'withdraw'), "BankAccount should have a withdraw method"
```

```
assert hasattr(result, 'get_balance'), "BankAccount should have a get_balance method"
assert result.get_balance() == 60, "Balance should be 60 after deposit 100 and withdraw 40"
```

Test your solution

```
# Test with new account
account2 = BankAccount()
assert account2.get_balance() == 0, "New account should start with balance 0"

account2.deposit(75)
assert account2.get_balance() == 75, "Balance should be 75 after deposit"

account2.withdraw(25)
assert account2.get_balance() == 50, "Balance should be 50 after withdraw"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 6: Vehicle Inheritance

Exercise Description

Create a `Vehicle` class with an attribute `speed` and a method `move`. Then, create a subclass `Car` that adds an attribute `fuel`.

Example:

```
v = Vehicle(60)
c = Car(100, "Full")
```

After creating the classes, create a `Vehicle` object with speed 60 and a `Car` object with speed 100 and fuel "Full". Store them in variables called `result1` and `result2`.

Your solution

```
class Vehicle:
```

```
    # step 1: define the constructor method that takes self and speed as parameters
```

```
    # step 2: assign the speed parameter to self.speed
```

```
    # step 3: define the move method that takes self as parameter
```

```
    # step 4: print a formatted string with "Moving at" and the speed
```

```
class Car:
```

```
    # step 5: inherit from Vehicle class using parentheses
```

```
    # step 6: define the constructor method that takes self, speed, and fuel as parameters
```

```
    # step 7: call the parent constructor using super().__init__(speed)
```

```
    # step 8: assign the fuel parameter to self.fuel
```

```
# step 9: create a Vehicle object with speed 60
```

```
# step 10: create a Car object with speed 100 and fuel "Full"
```

```
# step 11: store the objects in result1 and result2 variables
```

Test your solution

```
# Test the Vehicle class
```

```
assert hasattr(result1, 'speed'), "Vehicle should have a speed attribute"
```

```
assert hasattr(result1, 'move'), "Vehicle should have a move method"
```

```
assert result1.speed == 60, "Vehicle speed should be 60"
```

```
# Test the Car class
```

```
assert hasattr(result2, 'speed'), "Car should have a speed attribute"
```

```
assert hasattr(result2, 'fuel'), "Car should have a fuel attribute"
```

```
assert hasattr(result2, 'move'), "Car should inherit move method"
```

```
assert result2.speed == 100, "Car speed should be 100"
```

```
assert result2.fuel == "Full", "Car fuel should be Full"
```

Test your solution

```
# Test inheritance
assert isinstance(result2, Car), "result2 should be an instance of Car"
assert isinstance(result2, Vehicle), "result2 should also be an instance of Vehicle"
assert not isinstance(result1, Car), "result1 should not be an instance of Car"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 7: Person String Representation

Exercise Description

Modify Person to add `__str__` to return a readable representation.

Example:

```
p = Person("Bob", 25)
print(p) # Output: Bob, 25 years old
```

After creating the class, create a Person object with name "Bob" and age 25, and store it in a variable called result.

Your solution

```
class Person:
    # step 1: define the constructor method that takes self, name, and age as parameters
    # step 2: assign the name parameter to self.name
    # step 3: assign the age parameter to self.age

    # step 4: define the __str__ method that takes self as parameter
    # step 5: return a formatted string with name, comma, age, and "years old"

# step 6: create a Person object with name "Bob" and age 25
# step 7: store the Person object in the result variable
```

Test your solution

```
# Test the Person class
assert hasattr(result, 'name'), "Person should have a name attribute"
assert hasattr(result, 'age'), "Person should have an age attribute"
assert result.name == "Bob", "Person name should be Bob"
assert result.age == 25, "Person age should be 25"
assert hasattr(result, '__str__'), "Person should have a __str__ method"
```

Test your solution

```
# Test string representation
assert str(result) == "Bob, 25 years old", "String representation should be 'Bob, 25
```

```
# Test with different person
p2 = Person("Alice", 30)
assert str(p2) == "Alice, 30 years old", "String representation should be 'Alice, 30"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 8: Calculator Static Method

Exercise Description

Create a class Calculator with a static method add.

Example:

```
print(Calculator.add(5, 3)) # Output: 8
```

After creating the class, call `Calculator.add(5, 3)` and store the result in a variable called `result`.

Your solution

```
class Calculator:
    # step 1: use the @staticmethod decorator
    # step 2: define the add method that takes x and y as parameters (no self)
    # step 3: return the sum of x and y

    # step 4: call Calculator.add with arguments 5 and 3
    # step 5: store the result in the result variable
```

Test your solution

```
# Test the Calculator class
assert hasattr(Calculator, 'add'), "Calculator should have an add method"
assert result == 8, "Calculator.add(5, 3) should return 8"

# Test that it's a static method (can be called without instance)
assert Calculator.add(10, 20) == 30, "Calculator.add(10, 20) should return 30"
```

Test your solution

```
# Test with different values
assert Calculator.add(0, 0) == 0, "Calculator.add(0, 0) should return 0"
assert Calculator.add(-5, 5) == 0, "Calculator.add(-5, 5) should return 0"
assert Calculator.add(100, 200) == 300, "Calculator.add(100, 200) should return 300"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 9: Inventory Management System

Exercise Description

Create a class `Product` with attributes `name`, `price`, and `quantity`. Add a method `total_value` that calculates the total value of the product (`price * quantity`). Then, create an `Inventory` class that manages multiple `Product` objects. The `Inventory` class should have methods to:

- Add a product.
- Remove a product by name.
- Calculate the total inventory value.

Example:

```
p1 = Product("Laptop", 1500, 10)
p2 = Product("Phone", 800, 20)
inventory = Inventory()
inventory.add_product(p1)
```

```
inventory.add_product(p2)
print(inventory.total_value()) # Output: 31000
```

After creating the classes, create the example scenario above and store the inventory in a variable called result.

Your solution

```
class Product:
    # step 1: define the constructor method that takes self, name, price, and quantity
    # step 2: assign the name parameter to self.name
    # step 3: assign the price parameter to self.price
    # step 4: assign the quantity parameter to self.quantity

    # step 5: define the total_value method that takes self as parameter
    # step 6: return the product of self.price and self.quantity

class Inventory:
    # step 7: define the constructor method that takes self as parameter
    # step 8: initialize self.products as an empty list

    # step 9: define the add_product method that takes self and product as parameters
    # step 10: append the product to self.products list

    # step 11: define the remove_product method that takes self and name as parameter
    # step 12: create a new empty list called new_products
    # step 13: iterate through each product in self.products
    # step 14: if the product name is not equal to the given name, append it
    # step 15: assign new_products to self.products
```

```

    # step 16: define the total_value method that takes self as parameter
    # step 17: initialize total to 0
    # step 18: iterate through each product in self.products
    # step 19: add the product's total_value to total
    # step 20: return total

# step 21: create Product p1 with "Laptop", 1500, 10
# step 22: create Product p2 with "Phone", 800, 20
# step 23: create Inventory object
# step 24: add p1 to inventory
# step 25: add p2 to inventory
# step 26: store the inventory in result variable

```

Test your solution

```

# Test the Product class
p_test = Product("Test", 100, 5)
assert hasattr(p_test, 'name'), "Product should have a name attribute"
assert hasattr(p_test, 'price'), "Product should have a price attribute"
assert hasattr(p_test, 'quantity'), "Product should have a quantity attribute"
assert hasattr(p_test, 'total_value'), "Product should have a total_value method"
assert p_test.total_value() == 500, "Product total_value should be 500 (100 * 5)"

```

Test your solution

```

# Test the Inventory class
assert hasattr(result, 'products'), "Inventory should have a products attribute"
assert hasattr(result, 'add_product'), "Inventory should have an add_product method"
assert hasattr(result, 'remove_product'), "Inventory should have a remove_product method"
assert hasattr(result, 'total_value'), "Inventory should have a total_value method"

```

```
assert len(result.products) == 2, "Inventory should have 2 products"
assert result.total_value() == 31000, "Total inventory value should be 31000 (15000 + 16000)"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 10: Library System

Exercise Description

Create a Book class with attributes title, author, and is_checked_out (default to False). Add methods check_out and return_book to update is_checked_out. Create a Library class that manages a collection of books. The Library class should have methods to:

- Add a book.
- Check out a book by title (if it's available).
- Return a book by title.
- List all books and their status.

Example:

```
b1 = Book("1984", "George Orwell")
b2 = Book("To Kill a Mockingbird", "Harper Lee")
library = Library()
library.add_book(b1)
library.add_book(b2)
library.check_out("1984")
```

After creating the classes, create the example scenario above and store the library in a variable called `result`.

Your solution

```
class Book:
    # step 1: define the constructor method that takes self, title, and author as parameters
    # step 2: assign the title parameter to self.title
    # step 3: assign the author parameter to self.author
    # step 4: set self.is_checked_out to False

    # step 5: define the check_out method that takes self as parameter
    # step 6: if self.is_checked_out is False, set it to True and return True
    # step 7: otherwise return False

    # step 8: define the return_book method that takes self as parameter
    # step 9: set self.is_checked_out to False

class Library:
    # step 10: define the constructor method that takes self as parameter
    # step 11: initialize self.books as an empty list

    # step 12: define the add_book method that takes self and book as parameters
```

```

        # step 13: append the book to self.books list

    # step 14: define the check_out method that takes self and title as parameters
    # step 15: iterate through each book in self.books
        # step 16: if the book title matches and is not checked out, call book.check_out
    # step 17: return "Book not available" if not found or already checked out

    # step 18: define the return_book method that takes self and title as parameters
    # step 19: iterate through each book in self.books
        # step 20: if the book title matches and is checked out, call book.return_book
    # step 21: return "Book not found or not checked out" if not found

    # step 22: define the list_books method that takes self as parameter
    # step 23: iterate through each book in self.books
        # step 24: determine status based on is_checked_out ("Checked Out" or "Available")
        # step 25: print the book title and status

# step 26: create Book b1 with "1984" and "George Orwell"
# step 27: create Book b2 with "To Kill a Mockingbird" and "Harper Lee"
# step 28: create Library object
# step 29: add b1 to library
# step 30: add b2 to library
# step 31: check out "1984"
# step 32: store the library in result variable

```

Test your solution

```

# Test the Book class
b_test = Book("Test Book", "Test Author")
assert hasattr(b_test, 'title'), "Book should have a title attribute"
assert hasattr(b_test, 'author'), "Book should have an author attribute"

```

```
assert hasattr(b_test, 'is_checked_out'), "Book should have an is_checked_out attribute"
assert b_test.is_checked_out == False, "Book should start as not checked out"
assert hasattr(b_test, 'check_out'), "Book should have a check_out method"
assert hasattr(b_test, 'return_book'), "Book should have a return_book method"
```

Test your solution

```
# Test the Library class and scenario
assert hasattr(result, 'books'), "Library should have a books attribute"
assert len(result.books) == 2, "Library should have 2 books"
assert result.books[0].title == "1984", "First book should be 1984"
assert result.books[0].is_checked_out == True, "1984 should be checked out"
assert result.books[1].is_checked_out == False, "To Kill a Mockingbird should be available"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 11: Employee Management System with Inheritance

Exercise Description

Create a base class `Employee` with attributes `name`, `id`, and `salary`. Add a method `get_details` to display basic details. Create two subclasses `Manager` and `Developer` that inherit from `Employee`.

- `Manager` should have an additional attribute `team_size`.
- `Developer` should have an additional attribute `programming_language`. Each subclass should override the `get_details` method to display specific information.

Example:

```
m = Manager("Alice", 1, 75000, 5)
d = Developer("Bob", 2, 60000, "Python")
print(m.get_details())
print(d.get_details())
# Output: Manager: Alice, ID: 1, Salary: 75000, Team Size: 5
#           Developer: Bob, ID: 2, Salary: 60000, Language: Python
```

After creating the classes, create the example scenario above and store the manager and developer in variables called `result1` and `result2`.

Your solution

```
class Employee:
    # step 1: define the constructor method that takes self, name, id, and salary as
    # step 2: assign the name parameter to self.name
    # step 3: assign the id parameter to self.id
```

```

        # step 4: assign the salary parameter to self.salary

    # step 5: define the get_details method that takes self as parameter
    # step 6: return a formatted string with "Employee:", name, "ID:", id, "Salary:"
    def get_details(self):
        return f"Employee: {self.name}, ID: {self.id}, Salary: {self.salary}"

class Manager:
    # step 7: inherit from Employee class using parentheses
    # step 8: define the constructor method that takes self, name, id, salary, and team_size
    # step 9: call the parent constructor using super().__init__(name, id, salary)
    # step 10: assign the team_size parameter to self.team_size
    def __init__(self, name, id, salary, team_size):
        super().__init__(name, id, salary)
        self.team_size = team_size

    # step 11: define the get_details method that takes self as parameter (override)
    # step 12: return a formatted string with "Manager:", name, "ID:", id, "Salary:", and "Team Size:"
    def get_details(self):
        return f"Manager: {self.name}, ID: {self.id}, Salary: {self.salary}, Team Size: {self.team_size}"

class Developer:
    # step 13: inherit from Employee class using parentheses
    # step 14: define the constructor method that takes self, name, id, salary, and programming_language
    # step 15: call the parent constructor using super().__init__(name, id, salary)
    # step 16: assign the programming_language parameter to self.programming_language
    def __init__(self, name, id, salary, programming_language):
        super().__init__(name, id, salary)
        self.programming_language = programming_language

    # step 17: define the get_details method that takes self as parameter (override)
    # step 18: return a formatted string with "Developer:", name, "ID:", id, "Salary:", and "Programming Language:"
    def get_details(self):
        return f"Developer: {self.name}, ID: {self.id}, Salary: {self.salary}, Programming Language: {self.programming_language}"

# step 19: create Manager with "Alice", 1, 75000, 5
# step 20: create Developer with "Bob", 2, 60000, "Python"
# step 21: store the objects in result1 and result2 variables
result1 = Manager("Alice", 1, 75000, 5)
result2 = Developer("Bob", 2, 60000, "Python")

```

Test your solution

```

# Test the Employee base class
e_test = Employee("Test", 999, 50000)

```

```
assert hasattr(e_test, 'name'), "Employee should have a name attribute"
assert hasattr(e_test, 'id'), "Employee should have an id attribute"
assert hasattr(e_test, 'salary'), "Employee should have a salary attribute"
assert hasattr(e_test, 'get_details'), "Employee should have a get_details method"
```

Test your solution

```
# Test the Manager class
assert isinstance(result1, Manager), "result1 should be a Manager instance"
assert isinstance(result1, Employee), "result1 should also be an Employee instance (i
assert hasattr(result1, 'team_size'), "Manager should have a team_size attribute"
assert result1.get_details() == "Manager: Alice, ID: 1, Salary: 75000, Team Size: 5"

# Test the Developer class
assert isinstance(result2, Developer), "result2 should be a Developer instance"
assert isinstance(result2, Employee), "result2 should also be an Employee instance (i
assert hasattr(result2, 'programming_language'), "Developer should have a programming
assert result2.get_details() == "Developer: Bob, ID: 2, Salary: 60000, Language: Pyth
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 12: E-commerce Order System

Exercise Description

Create a Product class with attributes name and price. Create an Order class that manages a list of Product objects. The Order class should have methods to:

- Add a product to the order.
- Remove a product by name.
- Calculate the total order value.

Example:

```
p1 = Product("Laptop", 1000)
p2 = Product("Headphones", 50)
order = Order()
order.add_product(p1)
order.add_product(p2)
print(order.total_order_value()) # Output: 1050
```

After creating the classes, create the example scenario above and store the order in a variable called result.

Your solution

```
class Product:
    # step 1: define the constructor method that takes self, name, and price as param
```

```
        # step 2: assign the name parameter to self.name
        # step 3: assign the price parameter to self.price

class Order:
    # step 4: define the constructor method that takes self as parameter
    # step 5: initialize self.products as an empty list

    # step 6: define the add_product method that takes self and product as parameters
    # step 7: append the product to self.products list

    # step 8: define the remove_product method that takes self and name as parameters
    # step 9: create a new empty list called new_products
    # step 10: iterate through each product in self.products
    # step 11: if the product name is not equal to the given name, append it
    # step 12: assign new_products to self.products

    # step 13: define the total_order_value method that takes self as parameter
    # step 14: initialize total to 0
    # step 15: iterate through each product in self.products
    # step 16: add the product's price to total
    # step 17: return total

# step 18: create Product p1 with "Laptop" and 1000
# step 19: create Product p2 with "Headphones" and 50
# step 20: create Order object
# step 21: add p1 to order
# step 22: add p2 to order
# step 23: store the order in result variable
```

Test your solution

```
# Test the Product class
p_test = Product("Test", 100)
assert hasattr(p_test, 'name'), "Product should have a name attribute"
assert hasattr(p_test, 'price'), "Product should have a price attribute"
assert p_test.name == "Test", "Product name should be Test"
assert p_test.price == 100, "Product price should be 100"
```

Test your solution

```
# Test the Order class
assert hasattr(result, 'products'), "Order should have a products attribute"
assert hasattr(result, 'add_product'), "Order should have an add_product method"
assert hasattr(result, 'remove_product'), "Order should have a remove_product method"
assert hasattr(result, 'total_order_value'), "Order should have a total_order_value method"
assert len(result.products) == 2, "Order should have 2 products"
assert result.total_order_value() == 1050, "Total order value should be 1050 (1000 + 50)"
```

Debug your solution

The following cells contain code to generate links to Python Tutor for debugging your code when running the tests. **You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.**

```
from IPython.display import HTML;import urllib.parse as u;HTML('<a href="https://pyth
```

Exercise 13: School Grading System

Exercise Description

Create a Student class with attributes name, grades (a list of integers), and methods:

- add_grade to add a grade.
- average_grade to calculate the average grade. Create a Course class that manages a list of Student objects. The Course class should have methods to:
- Add a student.
- Display all students' names and their average grade.

Example:

```
s1 = Student("Alice", [90, 80, 85])
s2 = Student("Bob", [70, 75, 80])
course = Course()
course.add_student(s1)
course.add_student(s2)
course.display_students()
# Output: Alice - Average Grade: 85.00
#         Bob - Average Grade: 75.00
```

After creating the classes, create the example scenario above and store the course in a variable called result.

Your solution

class Student:

```
# step 1: define the constructor method that takes self, name, and grades with de
# step 2: assign the name parameter to self.name
# step 3: if grades is provided, assign it to self.grades, otherwise assign e

# step 4: define the add_grade method that takes self and grade as parameters
# step 5: append the grade to self.grades list

# step 6: define the average_grade method that takes self as parameter
# step 7: if self.grades is not empty, return the sum divided by length
# step 8: otherwise return 0
```

class Course:

```
# step 9: define the constructor method that takes self as parameter
# step 10: initialize self.students as an empty list

# step 11: define the add_student method that takes self and student as parameter
# step 12: append the student to self.students list

# step 13: define the display_students method that takes self as parameter
# step 14: iterate through each student in self.students
# step 15: print the student name, dash, "Average Grade:", and the average

# step 16: create Student s1 with "Alice" and grades [90, 80, 85]
# step 17: create Student s2 with "Bob" and grades [70, 75, 80]
# step 18: create Course object
# step 19: add s1 to course
# step 20: add s2 to course
# step 21: store the course in result variable
```

